

Exercise 1: Implementing the Singleton Pattern

Scenario:

You need to ensure that a logging utility class in your application has only one instance throughout the application lifecycle to ensure consistent logging.

Steps:

1. Create a New Java Project:

- Create a new Java project named **SingletonPatternExample**.

2. Define a Singleton Class:

- Create a class named `Logger` that has a private static instance of itself.
- Ensure the constructor of `Logger` is private.
- Provide a public static method to get the instance of the `Logger` class.

3. Implement the Singleton Pattern:

- Write code to ensure that the `Logger` class follows the Singleton design pattern.

4. Test the Singleton Implementation:

- Create a test class to verify that only one instance of `Logger` is created and used across the application.

Solution:

```
public class SingletonPattern {  
    Run | Debug  
    public static void main(String[] args) {  
        AppLogger loggerA = AppLogger.getLogger();  
        AppLogger loggerB = AppLogger.getLogger();  
  
        loggerA.writeLog(message: "Application Started");  
  
        System.out.println(loggerA == loggerB);  
    }  
}  
  
class AppLogger {  
    private static AppLogger instance;  
  
    private AppLogger() {  
        System.out.println(x: "Logger Object Created");  
    }  
  
    public static AppLogger getLogger() {  
        if (instance == null) {  
            instance = new AppLogger();  
        }  
  
        return instance;  
    }  
  
    public void writeLog(String message) {  
        System.out.println(message);  
    }  
}
```

Output:

```
PS C:\Users\2006p\OneDrive\Documents\DeepSkilling> javac SingletonPattern.java  
PS C:\Users\2006p\OneDrive\Documents\DeepSkilling> java SingletonPattern  
Logger Object Created  
Application Started  
true
```

Exercise 2: Implementing the Factory Method Pattern

Scenario:

You are developing a document management system that needs to create different types of documents (e.g., Word, PDF, Excel). Use the Factory Method Pattern to achieve this.

Steps:

1. Create a New Java Project:

- Create a new Java project named **FactoryMethodPatternExample**.

2. Define Document Classes:

- Create interfaces or abstract classes for different document types such as **WordDocument**, **PdfDocument**, and **ExcelDocument**.

3. Create Concrete Document Classes:

- Implement concrete classes for each document type that implements or extends the above interfaces or abstract classes.

4. Implement the Factory Method:

- Create an abstract class **DocumentFactory** with a method **createDocument()**.
- Create concrete factory classes for each document type that extends **DocumentFactory** and implements the **createDocument()** method.

5. Test the Factory Method Implementation:

- Create a test class to demonstrate the creation of different document types using the factory method.

Solution:

```
public class FactoryPattern {  
    public static void main(String[] args) {  
  
        DocumentFactory wordFactory = new WordFactory();  
        DocumentFactory pdfFactory = new PdfFactory();  
        DocumentFactory excelFactory = new ExcelFactory();  
  
        Document wordDoc = wordFactory.createDocument();  
        Document pdfDoc = pdfFactory.createDocument();  
        Document excelDoc = excelFactory.createDocument();  
  
        wordDoc.open();  
        pdfDoc.open();  
        excelDoc.open();  
    }  
}  
  
interface Document {  
    void open();  
}  
  
class WordDocument implements Document {  
  
    @Override  
    public void open() {  
        System.out.println("Opening Word Document");  
    }  
}
```

```
class PdfDocument implements Document {

    @Override
    public void open() {
        System.out.println("Opening PDF Document");
    }
}

class ExcelDocument implements Document {

    @Override
    public void open() {
        System.out.println("Opening Excel Document");
    }
}

abstract class DocumentFactory {

    public abstract Document createDocument();
}

class WordFactory extends DocumentFactory {

    @Override
    public Document createDocument() {
        return new WordDocument();
    }
}

class PdfFactory extends DocumentFactory {

    @Override
    public Document createDocument() {
        return new PdfDocument();
    }
}
```

```
}  
  
class ExcelFactory extends DocumentFactory {  
  
    @Override  
    public Document createDocument() {  
        return new ExcelDocument();  
    }  
}
```

Output:

```
PS C:\Users\2006p\OneDrive\Documents\DeepSkillig> javac FactoryPattern.java  
PS C:\Users\2006p\OneDrive\Documents\DeepSkillig> java FactoryPattern  
Opening Word Document  
Opening PDF Document  
Opening Excel Document
```